

Previsão do nível de adaptabilidade dos estudantes ao ensino remoto usando uma Rede Neural Single Layer

Caio Cardoso Dos Santos

11202021632

Joel da Silva Junior

11201920366

Victor Ravazio de Lima

11201920941

Computação Evolutiva e Conexionista

Professor: Mateus Coelho Silva

1 - Introdução

A evasão escolar representa um grande desafio em todo o Brasil, podendo causar bilhões em custo anual ao país devido suas implicações sociais, econômicas e de saúde pública [1], e com a tecnologia digital cada vez mais presente no processo de educação dos alunos [2], reforçado após o período de isolamento social durante a pandemia do Covid-19, no qual as plataformas para aprendizado *online* se tornarem o principal meio para continuidade dos estudos [3], notamos a **importância** que a adaptação para o estudo remoto possuem na continuidade da trajetória dos estudantes como um todo.

Com isso, esse projeto de Aprendizado de Máquina Supervisionado tem como **objetivo** utilizar Redes Neurais Artificiais para prever, com base em diversas características dos estudantes, qual será o seu nível de adaptabilidade ao ensino remoto, podendo-o classificar em três grupos distintos referente a essa adaptabilidade: “**Baixa**”, “**Moderada**” e “**Alta**”.

Dessa forma, as instituições escolares serão capazes de identificar quais alunos têm a tendência de possuírem mais dificuldade no estudo remoto, apenas baseado em suas características, tornando possível um plano de ação preventivo para reforçar o estudo desses alunos. Ademais, esse projeto poderá ser estendido para identificar quais características mais influenciam tais classificações, direcionando o reforço para contornar uma baixa adaptabilidade.

2 - Referencial Teórico

2.1 - Rede Neural Artificial

Uma Rede Neural Artificial (RNA) trata-se de um modelo computacional, inspirado no funcionamento do cérebro humano, desenvolvido para reproduzir o comportamento humano em tarefas como associação, aprendizado, generalização, tolerância a falhas e abstração. A RNA é constituída por unidades de processamento simples, chamados neurônios artificiais, responsáveis por calcular funções matemáticas [4].

Em sua essência, uma RNA recebe um sinal de entrada que atravessa uma camada de processamento (denominada oculta) composta por uma ou mais unidades de processamento treinadas (neurônios artificiais). Estes neurônios processam as informações

recebidas e transmitem o sinal resultante para a camada de saída. Caso existam muitas subcamadas na camada oculta, então a RNA é denominada profunda [5].

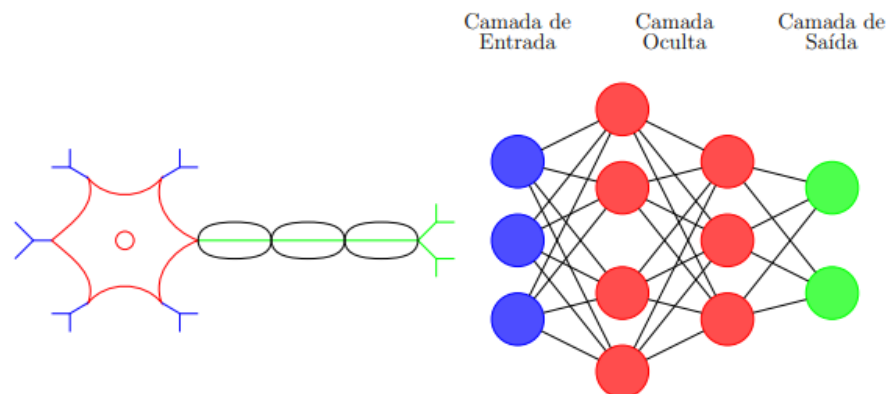


Figura 1 - Neurônio Biológico x Rede Neural Artificial [5]

2.2 - Perceptron de Rosenblatt

Apresentado por Frank Rosenblatt em 1958, o *Perceptron* é uma rede neural de um único neurônio cujo objetivo era classificar um vetor de entradas X em duas classes através do hiperplano [4]. Segue a representação estrutural do neurônio artificial (*Perceptron*) de Rosenblatt.

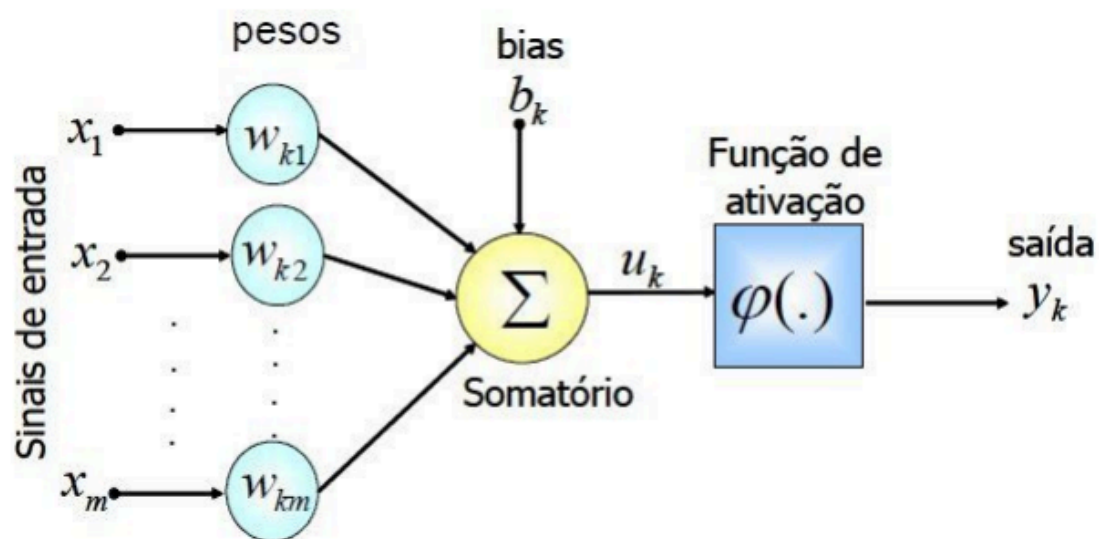


Figura 2 - Perceptron de Rosenblatt

Sua formulação matemática é dada por:

$$X = [X_{k1} \ X_{k2} \dots X_{km} \ 1] \quad (1)$$

$$W = [W_{k1} \ W_{k2} \dots W_{kn} \ b_k] \quad (2)$$

$$v_k = X \cdot W^T \quad (3)$$

$$\hat{Y} = \varphi(v_k) \quad (4)$$

Como ilustrado na figura e fórmulas acima, o modelo recebe um vetor X de “ m ” variáveis de entrada em conjunto com um elemento “1” correspondente ao *bias*. A rede então realiza uma soma das entradas ponderadas pelos pesos W_{ki} (que são inicializados com valores aleatórios), resultando em v_k . Este resultado então passa por uma função de ativação $\varphi(v_k)$ que atua como um decisor sobre a ativação ou não do neurônio. Para o caso do Perceptron, a função de ativação utilizada é a função degrau, expressa em (5).

$$\varphi(v_k) = 0 \text{ se } v_k < 0, \text{ ou } 1 \text{ se } v_k \geq 0 \quad (5)$$

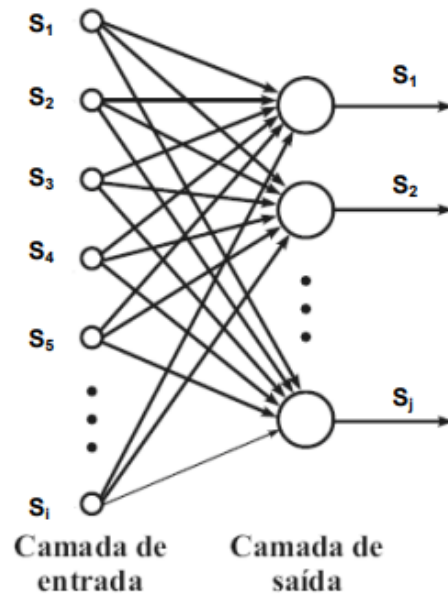
No *Perceptron*, utiliza-se para o treinamento um modelo de aprendizado supervisionado, onde já existe um conhecimento prévio dos rótulos “ Y ” para cada vetor de entrada X . Especificamente, a técnica de aprendizado utilizada neste modelo é o Aprendizado Hebbiano, cuja a atualização dos pesos é expresso na equação (6)

$$\Delta W_{ij} = \eta \cdot (Y_j - \hat{Y}_j) \cdot X_i \quad (6)$$

Onde η é a taxa de aprendizado, $(Y_j - \hat{Y}_j)$ é o da previsão (diferença entre o rótulo real e o previsto), e X_i é a entrada.

2.3 - Rede Neural Single Layer

Uma Rede Neural *Single Layer* (ou camada única) consiste em um modelo simples de RNA onde os neurônios são organizados em uma única camada, de forma que a saída de cada neurônio constitui uma saída da rede [6]. Segue, na figura, uma representação da estrutura de uma Rede Neural *Single Layer*.



Sua formulação matemática é similar às expressões apresentadas para o *Perceptron* de Rosenblatt, com a diferença que agora, com a adição de mais neurônios, temos uma matriz de pesos com um número de linhas igual ao número de neurônios. Em outras palavras, para uma RNA *Single Layer* com “m” Neurônios e “n” entradas, temos as seguintes expressões :

$$W_1 = [W_{11} \ W_{12} \dots \ W_{1n} \ b_1]$$

$$\vdots$$

$$W_m = [W_{m1} \ W_{m2} \dots \ W_{mn} \ b_m]$$

$$\hat{Y}_1 = \varphi(X \cdot W_1^T)$$

$$\vdots$$

$$\hat{Y}_m = \varphi(X \cdot W_m^T)$$

Onde cada $\hat{Y}_i$ corresponde ao resultado previsto pelo neurônio “i”.

Uma função de ativação muito utilizada para este modelo é a função sigmoide, expressa na equação (7)

$$\varphi(S) = \frac{1}{1+e^{-s}} \quad (7)$$

2.4 - Backpropagation e Gradiente descendente

O gradiente descendente consiste em um método iterativo utilizado para minimizar uma função de interesse “f”. Em sua essência, buscamos encontrar um valor para um parâmetro β que minimize sua respectiva função [5]. No caso de uma RNA, o interesse está em minimizar a função de erro na rede, através justamente do ajuste dos pesos W_{ij} . Seja então ε uma função de erro escolhida expresso em (8), a equação (9) denota o cálculo do gradiente descendente para uma RNA.

$$\varepsilon = (\hat{Y} - Y)^2 \quad (8)$$

$$W_{ij} = W_{ij} - \eta \cdot \frac{\partial \varepsilon}{\partial W_{ij}} \quad (9)$$

Os pesos então são atualizados desta forma até que se atinja a convergência (mínimo global para ε).

O Backpropagation é um modelo refinado para se obter métricas sobre como o valor de cada peso influencia no erro da RNA. A técnica busca propagar o erro de trás para frente numa RNA aplicando a regra da cadeia para calcular o gradiente do erro (vetor de derivadas) em função dos pesos W_{ij} . Este gradiente fornece ao gradiente descendente quais parâmetros deverão ser ajustados e em qual direção [7]. No caso de uma RNA *Single Layer*, cuja função de ativação é sigmoide (7) e o erro é quadrático (8), a formulação do gradiente do erro é dado por:

$$\frac{\partial \varepsilon}{\partial W_{ij}} = \frac{\partial \varepsilon}{\partial \hat{Y}} \cdot \frac{\partial \hat{Y}}{\partial S} \cdot \frac{\partial S}{\partial W_{ij}} \quad (10)$$

$$\frac{\partial \varepsilon}{\partial \hat{Y}} = 2 \cdot (\hat{Y} - Y) \quad (11)$$

$$\frac{\partial \hat{Y}}{\partial S} = \varphi(S) \cdot (1 - \varphi(S)) \quad (12)$$

$$\frac{\partial S}{\partial W_{ij}} = X_j \quad (13)$$

$$W_{ij} = W_{ij} - \eta \cdot 2 \cdot (\hat{Y} - Y) \cdot \varphi(S) \cdot (1 - \varphi(S)) \cdot X_j \quad (14)$$

3 - Metodologia

3.1 - A Base de dados

A base de dados selecionada [8] foi retirada da plataforma *Kaggle* e provém de uma pesquisa realizada entre dezembro de 2020 e fevereiro de 2021, durante a pandemia do Covid-19, com 1205 estudantes de diferentes níveis de ensino no país de Bangladesh. [9]

A fim de prever o nível de adaptação dos alunos com o ensino online devido à pandemia, foram levantados 14 atributos. São eles: gênero, idade, nível de escolaridade, tipo de instituição (pública ou privada), aluno cursa ou não TI, mora ou não na cidade, nível de racionamento de energia, condição socioeconômica da família, tipos de conexão com internet (dados móveis ou *wi-fi*), qualidade da rede (2G, 3G ou 4G), duração das aulas, tipo de dispositivo usado nas aulas, existência ou não de uma plataforma de aprendizagem própria da instituição e o nível de adaptação do aluno (baixa, moderada ou alta).

Para a realização do projeto foi utilizada a linguagem *Python* e as bibliotecas *Pandas*, *Numpy* e *Matplotlib* a fim de carregar e manipular os dados do *dataset*, estruturar a RNA e realizar o seu treinamento, além de criar os gráficos para compreensão dos resultados obtidos.

O pré-processamento dos dados antes do treinamento dividiu-se em 5 etapas. Inicialmente os dados foram embaralhados para não enviesar o aprendizado da RNA. Agrupou-se então 13 categorias para compor o vetor X e a coluna *Adaptivity Level* foi separada para ser o rótulo Y desejado.

As categorias nominais binárias *Gender*, *Institution Type*, *IT Student*, *Location*, *Load-shedding*, *Internet Type* e *Self Lms* foram adaptadas para representarem sua dualidade com 0 ou 1.

As variáveis nominais multiclases precisaram de outro tratamento. Em *Education Level* e *Device* aplicou-se a técnica *one-hot encoding*, em que é criado uma nova coluna para cada valor possível de uma categoria, preenchendo-a com 1 se ocorre e 0 nas demais colunas pertencentes à respectiva categoria.

As categorias multiclases ordinais *Financial Condition*, *Network Type*, *Age* e *Class Duration* utilizaram a função *map* para manter a hierarquia entre os valores correspondentes de cada intervalo. Como *Age* e *Class Duration* representam uma faixa de valores, cada uma de ambas as categorias também deve seguir a lógica acima [10]. Optou-se então por preencher cada intervalo pelo valor médio de seus respectivos limites, antes da aplicação da função *map*.

Por fim, a coluna alvo nominal multiclasse *Adaptivity Level* também recebeu a aplicação da técnica *one-hot encoding*, a fim de criar uma nova coluna para cada nível de adaptação, preenchendo-a com 1 se ocorre e 0 nas demais colunas referentes ao rótulo Y.

Para a estruturação da RNA, criou-se uma Rede Neural *Single Layer* de 3 neurônios, uma para cada nível de adaptação, 17 entradas (agrupando as 13 características do vetor X, os

3 níveis de adaptação e a coluna referente ao bias), além da escolha da função sigmoide (7) como de ativação.

3.2 - O Treinamento

O treinamento da RNA desenvolvida baseou-se nas técnicas discutidas ao longo do referencial teórico, em destaque aqui o uso do *Backpropagation* para calcular o gradiente do erro e do método gradiente descendente para realizar a atualização dos pesos da rede a cada iteração.

Inicialmente, os dados foram divididos em 75% para treinamento e 25% para testes. A taxa de aprendizado foi definida em 0.01 e o número de épocas em 1000. A cada época, o algoritmo percorre todos os dados do conjunto de treino, calculando a saída prevista ($\hat{Y}_i$) por meio da aplicação de uma função de aplicação sigmóide (7) sobre a combinação linear das entradas ponderadas pelos pesos (3). O erro é calculado como a diferença entre os valores previstos e reais. Em seguida é aplicado o gradiente descendente (equação (14)) para atualizar cada um dos pesos W_{ij} da rede.

A fim de acompanhar o comportamento do erro ao longo do treinamento, e buscando evitar *overfitting*, o modelo é avaliado ao final de cada época por meio das métricas de Erro Quadrático Médio (MSE) e a Raíz do Erro Quadrático Médio (RMSE), explicadas a seguir, utilizando o conjunto de teste como validação. Estas métricas de avaliação são discutidas com mais detalhes na subseção seguinte.

Nós observamos o comportamento da curva de **convergência do erro** entre o **resultado real e resultado previsto** pela RNA dos dados de teste (eixo y) com o decorrer das **épocas** de treinamento (eixo x), representadas tanto como o modelo do **Erro Quadrático Médio** (MSE), por utilizarmos função de erro quadrática no treinamento do modelo, quanto pela **Raiz do Erro Quadrático Médio** (RMSE), para normalizar a escala do problema:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$
$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Os gráficos foram gerados utilizando a biblioteca **matplotlib** no Python.

3.3 - Pós Treinamento e Avaliação

Terminado o treinamento, será testado o desempenho da RNA com os 25% dos dados não utilizados na etapa anterior. Para avaliar sua predição, por se tratar de um problema de classificação, a análise será baseada nos resultados obtidos a partir da **Matriz de Confusão**, no qual os dados reais são cruzados dos resultados da base de teste com os resultados obtidos pela predição da rede neural.

Com ela obtemos quatro tipos de cruzamentos:

- **Verdadeiros Positivos (TP):** O modelo previu a classe positiva corretamente.
- **Verdadeiros Negativos (TN):** O modelo previu a classe negativa corretamente.
- **Falsos Positivos (FP):** O modelo previu a classe positiva, mas a resposta real era negativa.
- **Falsos Negativos (FN):** O modelo previu a classe negativa, mas a resposta real era positiva.

Para cada uma das classificações, teremos um tipo de classificação diferente dos dados, sendo eles:

- Analisando a classificação **“Baixa”**:

Reais:	Baixa	TP	FN	FN
	Moderada	FP	TN	TN
	Alta	FP	TN	TN
		Baixa	Moderada	Alta
		Previstos:		

- Analisando a classificação **“Moderada”**:

Reais:	Baixa	TN	FP	TN
	Moderada	FN	TP	FN
	Alta	TN	FP	TN
		Baixa	Moderada	Alta
		Previstos:		

- Analisando a classificação **“Alta”**:

Reais:	Baixa	TN	TN	FP
	Moderada	TN	TN	FP
	Alta	FN	FN	TP
		Baixa	Moderada	Alta
		Previstos:		

A partir desses cruzamentos, conseguimos calcular as métricas para análise, sendo elas:

- **Acurácia global:**
Mede a proporção geral de acertos do modelo em relação ao total de previsões.

$$Acurácia = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precisão:**

Avalia a quantidade de falsos positivos.

$$Precisão = \frac{TP}{TP + FP}$$

- **Revocação:**

Avalia a quantidade de falsos negativos.

$$Revocação = \frac{TP}{TP + FN}$$

- **F1-Score:**

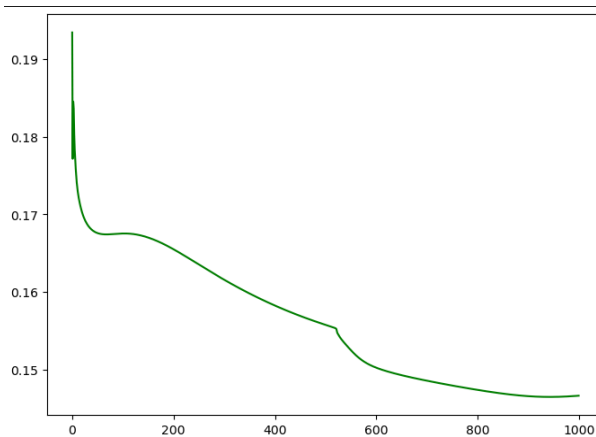
É a média harmônica entre a Precisão e a Revocação. Ele busca encontrar um equilíbrio entre as duas métricas.

$$F1Score = 2 * \frac{Precisão * Revocação}{Precisão + Revocação}$$

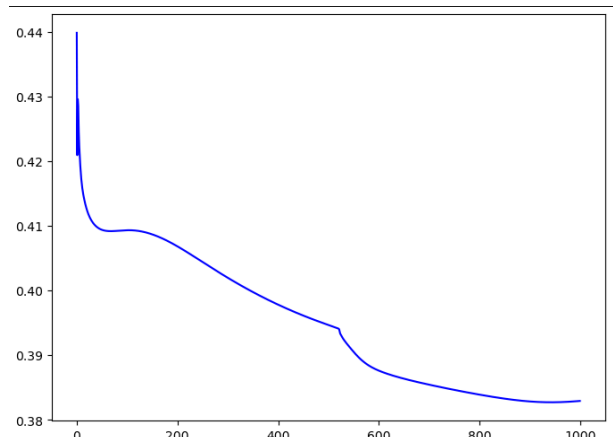
3 - Resultados e Discussão:

A partir disso, obtemos os seguintes resultados para a curva de **convergência do erro** entre o **resultado real** e **resultado previsto** pela RNA dos dados de teste (eixo y) com o decorrer das **épocas** de treinamento (eixo x).

Distribuição do **Erro Quadrático Médio** pela quantidade de **Épocas**:

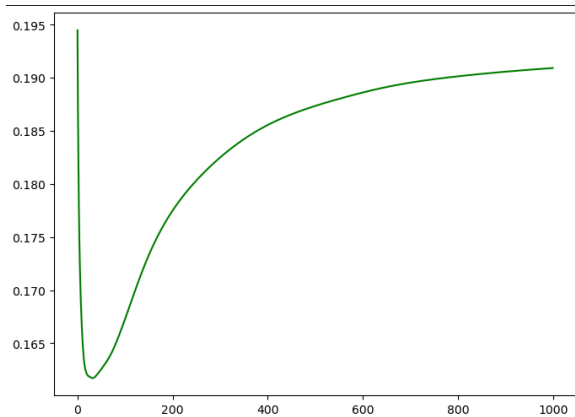


Distribuição da **Raiz do Erro Médio Quadrático** pela quantidade de **Épocas**:

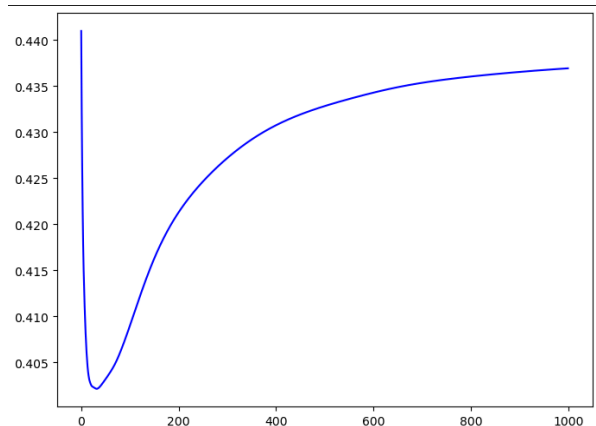


Nessa execução, observamos nas curvas um comportamento **satisfatório** de redução do erro conforme o treinamento ocorre, porém, os resultados podem variar, com algumas execuções demonstrando um comportamento diferente desse padrão de redução, como exemplificado a seguir:

Distribuição do **Erro Quadrático Médio** pela quantidade de **Épocas**:



Distribuição da **Raiz do Erro Médio Quadrático** pela quantidade de **Épocas**:



Isso pode indicar um comportamento **indesejado**, com uma possível predição inconsistente, em que algumas execuções reduzem bem o erro, e em outras não. Os dados utilizados para o cálculo das métricas são referentes aos dois primeiros gráficos demonstrados, após ser observado que os valores das métricas são semelhantes em ambos os casos.

Como comentado na seção de Metodologia, distribuímos nossos dados em uma Matriz de Confusão para gerar as métricas de análise:

Reais:	Baixa	84	40	0
	Moderada	30	123	4
	Alta	1	16	4
		Baixa	Moderada	Alta
		Previstos:		

A partir desses resultados, conseguimos calcular as métricas para análise, sendo elas:

- **Acurácia global:** 0.6986754966887417
- **Precisão:**
 - Precisão da classe “Baixa”: 0.7304347826086957
 - Precisão da classe “Moderada”: 0.6871508379888268
 - Precisão da classe “Alta”: 0.5
- **Revocação:**
 - Revocação da classe “Baixa”: 0.6774193548387096
 - Revocação da classe “Moderada”: 0.7834394904458599
 - Revocação da classe “Alta”: 0.19047619047619047
- **F1-Score:**
 - F1-Score da classe “Baixa”: 0.702928870292887
 - F1-Score da classe “Moderada”: 0.7321428571428572
 - F1-Score da classe “Alta”: 0.27586206896551724

A classe “Moderada” foi a mais equilibrada dentre as três, com um desempenho na métrica F1-Score de 0.73. Ela é a que mais possui dados distribuídos na base, o que pode contribuir para sua predição ser mais assertiva.

A classe “Baixa” vem com um equilíbrio um pouco mais abaixo (0.70), o que é justificado principalmente por sua revocação mais baixa quando comparada com a classe “Moderada”. Isso é prejudicial pois é extremamente importante para o nosso modelo que se capture corretamente a classe de adaptabilidade “Baixa”, justamente para se ter um plano de ação específico para essa classe de alunos e sermos capazes de possibilitar que eles aumentem seu grau de adaptabilidade, por isso a taxa de “Falsos Negativos” deve ser a menor possível.

Observamos o baixíssimo desempenho obtido pela classificação “Alta” (< 0.27), o que indica que tanto sua Precisão quanto sua Revocação estão desequilibrados e com valores baixos, como observamos acima, dessa forma concluímos que o modelo não prevê bem o resultado dessa classe.

Isso ocorre porque, segundo o próprio trabalho de pesquisa realizado na captura dos dados em [9], reparamos que os dados de treinamento estão **desbalanceados**, com poucos representativos da classe “Alta” (menos de 9% dos dados totais). Existe a possibilidade disso ser representativo de uma distribuição do mundo real, mas isso atrapalha a predição de nosso sistema de forma correta, fazendo-se necessário um maior balanceamento da base de dados.

Além disso, outra alternativa é de que a rede neural sendo utilizada não é complexa o suficiente para prever o resultado, por se tratar de uma rede “single layer” (uma camada de neurônios artificiais), capturando apenas relações lineares, além da função de erro utilizada poder também não ser suficiente para fazer a atualização desse modelo. É importante notar que a diferença de escala entre diferentes tipos de dados também pode atrapalhar a predição correta, fazendo-se necessário a normalização dos dados.

Esses fatores combinados podem justificar as métricas razoavelmente baixas por todas as distribuições, com uma acurácia global de 0.70. Precisamos modificar o modelo e explorar opções de solução que capturem essa relação mais complexa entre as características e o resultado.

4 - Conclusão:

A partir dos resultados obtidos, podemos concluir que a RNA desenvolvida apresentou um desempenho moderado, principalmente se tratando de um modelo simples (*Single Layer*) operando sob um conjunto de dados dotado de classes desbalanceadas. Com uma acurácia global em torno de 0.70 o modelo demonstra-se ser capaz de capturar padrões relevantes nos dados.

O modelo possui um desempenho satisfatório na previsão das classes “baixa” e “moderada”, apresentando valores equilibrados de precisão, revocação e F1-score. Entretanto, a classe “alta” apresenta baixos valores quanto à revocação e F1-Score, possível consequência da natureza desbalanceada dos dados ou menor presença desta

classe no conjunto de treino. Por outro lado, a queda nas curvas de erro ao longo das épocas indica que o modelo está realmente aprendendo, sugerindo convergência.

Para aprimorar o desempenho, seria necessário a utilização de uma base de dados mais balanceada, especialmente com maior representatividade da classe “alta”, fazendo com que o modelo consiga relacionar de maneira mais confiável as características que levam a essa classificação.

Além disso, é importante considerar o aumento da complexidade da estrutura da rede neural utilizada, por meio da adição de mais camadas, a fim de melhorar a sua capacidade de lidar com relações não lineares mais complexas presentes nos dados. Outra estratégia seria a reavaliação das funções, hiperparâmetros ou até mesmo a inserção de métricas heurísticas visando um modelo mais robusto e com melhor capacidade de generalização.

5 - Declaração de ética (IA Generativa):

Declaramos total autoria e responsabilidade pelo conteúdo do projeto aqui presente, com ferramentas de IA generativa sendo utilizadas apenas para padronizar o formato textual das referências, e na tradução de artigos em inglês para melhor compreensão. Nenhum uso foi feito na realização tanto do código, quanto do conteúdo do relatório. O código foi baseado no material prático desenvolvido em sala de aula, e os dados utilizados foram publicamente disponibilizados em [8].

Referências:

[1] FERREIRA, Sergio Guimarães; RIBEIRO, Giovanna; TAFNER, Paulo. Abandono e evasão escolar no Brasil. Instituto Mobilidade e Desenvolvimento Social, 2022. p. 1-40. Disponível em: https://imdsbrasil.org/doc/ImdsNT001-2022-AbandonoEvas%C3%A3oEscolarBrasil_1.pdf.

Acesso em: 17 mar. 2026.

[2] BITTENCOURT, Priscilla Aparecida Santana; ALBINO, João Pedro. O uso das tecnologias digitais na educação do século XXI. Revista Ibero-Americana de Estudos em Educação, v. ?, p. 205-214, 2017. Disponível em: <https://periodicos.fclar.unesp.br/iberoamericana/article/view/9433/6260>. Acesso em: 17 mar. 2026.

[3] CORDEIRO, Karolina Maria de Araújo. O impacto da pandemia na educação: a utilização da tecnologia como ferramenta de ensino. 2020. Disponível em: https://d1wqtxts1xzle7.cloudfront.net/76871647/O_IMPACTO_DA_PANDEMIA_NA_EDUCACAO_A_UTILIZACAO_DA_TECNOLOGIA_COMO_FERRAMENTA_DE_ENSINO-libre.pdf?1639983390=&response-content-disposition=inline%3B+filename%3DO_IMPACTO_DA_PANDEMIA_NA_EDUCACAO_A_UTIL.pdf&Expires=1773794566&Signature=Nl~PUS5zj~w4zdHTXYKilulOY42oKnuaf4LgkJgpckdYw8qVrE-uHjxo-brLculFRlr1F8JHdAzryWH7a6LNgw9eT4mvCjH6Li--KTIA37ckd74HHg~hfk1hyC2gctI9Jd3KVNJKLuDmF6VoLE1dH94ZbW2NGFkB Aurml~w6g5ydewGYThVXE7pOT2l0XNidHUslrIvrr8RGstC4gqIKoi91gQBkHn7xxHiAJ26Tnlga-lzGn3kQnt2oNm77dSUMFz21g1Q-v6-BzwAorQnZ0HHdDLh-jpROSzpUKb1r15NkBhVd

[CWrgjccWL7pSejUhnsEGavioFowQJu5N1PT-AA__&Key-Pair-Id=APKAJLOHF5GGSLRBV4ZA](#). Acesso em: 17 mar. 2026.

[4] MARTINIANO, A.; FERREIRA, R. P.; FERREIRA, A.; FERREIRA, A.; SASSI, R. J. Using an artificial neural network to approximate the temporal evolution function of the lorenz system. Revista Produção e Desenvolvimento, v. 2, n. 1, p. 26-38, abr. 2016. Disponível em: <https://revistas.cefet-rj.br/index.php/producaoedesenvolvimento/article/view/e94>. Acesso em: 17 mar. 2026.

[5] PINHEIRO, Maquise de Medeiros. Perceptron multicamadas: uma ferramenta de aprendizado supervisionado. 2021. 44 f. Trabalho de Conclusão de Curso (Graduação em Estatística) - Instituto de Matemática e Estatística, Universidade Federal Fluminense, Niterói, 2021.

[6] ORTEGA, Gustavo Victor Chávez. Redes neurais na identificação de perdas comerciais do setor elétrico. 2008. Dissertação (Mestrado em Engenharia Elétrica) - Pontifícia Universidade Católica do Rio de Janeiro, Rio de Janeiro, 2008.

[7] BERGMANN, Dave; STRYKER, Cole. What is backpropagation? São Paulo: IBM, 2024. Disponível em: <https://www.ibm.com/br-pt/think/topics/backpropagation>. Acesso em: 17 mar. 2026.

[8] SUZAN, Md Mahmudul; SAMRIN, Nishat; BISWAS, Al Amin; PRAMANIK, Md. Students adaptability level in online education. 2026. Disponível em: <https://www.kaggle.com/datasets/mdmahmudulhasansuzan/students-adaptability-level-in-online-education>. Acesso em: 17 mar. 2026.

[9] SUZAN, Md Mahmudul; SAMRIN, Nishat; BISWAS, Al Amin; PRAMANIK, Md. Students' adaptability level prediction in online education using machine learning approaches. 2021. p. 1-7. DOI: 10.1109/ICCCNT51525.2021.9579741. Disponível em: https://www.researchgate.net/publication/355891881_Students'_Adaptability_Level_Prediction_in_Online_Education_using_Machine_Learning_Approaches. Acesso em: 17 mar. 2026.

[10] SHANTO, S. S.; JONY, A. I. Interpretable ensemble learning approach for predicting student adaptability in online education environments. Knowledge, v. 5, n. 2, p. 10, 2025. Disponível em: <https://www.mdpi.com/2673-9585/5/2/10>. Acesso em: 17 mar. 2026.